

Exercise 6: Draft with Claude, Grounded by RAG

DEV CYCLE	Develop (AI integration + retrieval)
CC FEATURES	anthropic SDK, keys via .env, the build loop on an AI call, prompt iteration, structured JSON output
SPEC TRACES	FR-05, FR-06, FR-13, FR-14; AI-01...AI-06
TIME	45 min
YOU PRODUCE	app/rag.py, app/drafting.py, app/safety.py and a draft endpoint that produces grounded, validated, flagged patient summaries

Objective

This is the heart of the day: using Claude Code to build a feature that itself calls Claude. You will replace the stub drafter with a real Claude API call and ground it with retrieval over uploaded reference summaries (RAG). Treat the model like any other dependency: key in .env, a typed function, strict output validation, graceful failure, and prompt iteration until the output is reliable. And treat it like an unreliable colleague: the model writes prose, the database supplies the facts and the doses.

Heads-up: in this exercise the prompt is built the simple way, by formatting record fields into the instruction text. That is intentionally unsafe (SEC-02, SEC-03 are still open). In Exercise 7 you will attack it and then fix it.

Walk-through

1 Check the SDK and key

The SDK is already installed from requirements.txt and your key is in .env from Exercise 2. Confirm both before debugging anything else.

RUN IN TERMINAL

```
PS> pip show anthropic
PS> git check-ignore .env
```

VALIDATE `pip show` prints a version. `git check-ignore .env` prints `.env`, which proves it is ignored. Your key is a secret like a password: never paste it into a prompt, a file other than `.env`, or a screenshot.

2 Build reference upload and chunking

RAG starts with getting good text in. The spec asks for heading-aware chunks and a dependency-free retriever, so nobody has to install a vector database for a PoC.

TYPE THIS PROMPT INTO CLAUDE CODE

Create `app/rag.py` implementing FR-13 and FR-14 with no new dependencies: `chunk_markdown(text)` splits on markdown headings and paragraphs into chunks of roughly 120 to 200 words, never over 1500 characters, and prefixes each chunk with its heading; a small BM25 scorer (lowercase, simple tokeniser, stop-word list) with `search(db, query, k=3)` returning chunk ids, document titles and scores. Add `POST /references` (multipart UploadFile .md or .txt only, 415 otherwise; 413 over 200 KB; stores document and chunks; requires X-API-Key) and `GET /references` (documents with chunk counts) to `app/routes.py`. Docstrings cite the FR IDs.

VALIDATE The diff adds app/rag.py with pure-Python BM25 (no numpy, no new package) and two routes. Then, in Swagger, upload the three files in `samples/references/`: each returns 201 with a `chunk_count` between 5 and 10. Try uploading a .pdf: 415.

SCREENSHOT SLOT *Swagger: POST /references returning chunk_count for the appendectomy reference*

3 Prove retrieval before you trust it

A retriever that returns plausible-looking wrong chunks is worse than none, because the model will confidently use them. Check it in isolation first.

TYPE THIS PROMPT INTO CLAUDE CODE

Without adding files to the app, write and run a one-off snippet that calls `app.rag.search` for the query "Acute appendicitis Laparoscopic appendectomy" and then for "Community-acquired pneumonia Type 2 diabetes mellitus", printing the top 3 chunks' document titles, headings and scores.

VALIDATE The appendicitis query returns only appendectomy chunks; the pneumonia query returns pneumonia chunks, likely including "Other health conditions" for the diabetes term. The heart-failure document should not appear in either top 3. If it does, ask Claude why (tokeniser? stop words?) and fix the scorer, not the test data.

4 Build the drafting module

Now the Claude call. You are asking for one typed function with a strict output contract, a single retry, and a safe failure. Be explicit about what the model is allowed to produce.

TYPE THIS PROMPT INTO CLAUDE CODE

Create `app/drafting.py` per AI-01 to AI-06. `generate_draft(record: dict, medications: list[dict], chunks: list[dict]) -> DraftResult` calls the Anthropic Messages API with the model from `ANTHROPIC_MODEL` (default `claude-sonnet-4-6`), `max_tokens 1500`, `timeout 30 s`, loading the key with `python-dotenv`. System prompt: you write discharge summaries for patients at a reading age of about 12; use only facts from the discharge record; use the reference material only for structure and wording; never write doses, frequencies or durations; return JSON only matching this schema: [paste SPEC §7.2]. For now, build the user message by formatting the record fields and chunks into text. Parse with `DraftContent.model_validate_json`. On invalid output retry once, then raise `DraftingError("ai_invalid_output")`. On API or network errors raise `DraftingError("ai_unavailable")`. Return content, citations (chunk ids), model, `prompt_version "draft-v1"` and token usage. Log errors without record contents.

VALIDATE Read before approving: `anthropic.Anthropic()` with a timeout; `messages.create(...)` with `model`, `max_tokens` and `system`; `load_dotenv()` before the environment is read; pydantic validation with exactly one retry; two distinct error codes; nothing hard-coded. If Claude "helpfully" parses doses out of the model output, reject it: doses come from the database.

5 Add the safety checks and wire it in

The model is fluent and sometimes wrong. Two cheap local checks catch the most dangerous kinds of wrong before a reviewer ever sees the draft.

TYPE THIS PROMPT INTO CLAUDE CODE

Create `app/safety.py`: `unknown_medications(draft_text, medications)` returns medicine names mentioned in the prose that are not in the record (check against a small list of common drug names plus the record's own names); `reading_grade(text)` computes Flesch-Kincaid grade locally with a simple syllable counter (AI-05). Then in `POST /discharges/{id}/drafts` replace `stub_draft`: retrieve the top 3 chunks with `rag.search` on `primary_diagnosis` plus procedures, call `generate_draft`, store citations, `reading_grade x10`, `safety_flags` (`unknown_medication:<name>`, `reading_grade_high`, `no_grounding`) and

provenance. On DraftingError store a generation_failed draft with the error_code, leave the record status unchanged, and return 503 {detail, draft_id} (FR-06). Remove the TODO(Ex6).

VALIDATE Restart uvicorn (it must re-read .env). First prove the failure path: set ANTHROPIC_MODEL=claude-does-not-exist in .env, restart, submit the pneumonia record and POST a draft for it. You get 503 with a draft_id; that draft shows generation_failed / ai_unavailable and the record is still submitted. Put the model name back, restart, and POST again: 201, version 2, with real plain-language sections, citations pointing at pneumonia chunks, a reading_grade, and a medication_table identical to the record. The model never touched a dose.

SCREENSHOT SLOT *A real draft: citations, reading_grade and the DB-rendered medication_table*

6 Engineer the prompt with evidence

Prompt engineering is a development task with measurable outcomes, not vibes. Iterate on one metric, reading grade, and keep the evidence.

TYPE THIS PROMPT INTO CLAUDE CODE

Look at the last draft's reading_grade and sections. Revise the system prompt in app/drafting.py to bring the reading grade down: short sentences, everyday words, expand every abbreviation (POD, PO, SOB, CXR), and say what each medicine is for without numbers. Bump prompt_version to "draft-v2". Then request a new draft (first reject the pending one as dr.rao with a reason; dr.iyer submitted it, so dr.iyer cannot) and compare the two reading grades.

VALIDATE The v2 draft has a lower reading_grade and no clinical abbreviations. Both drafts are kept, with their prompt_version, so you can show the improvement. You just ran your first prompt A/B test against a spec metric (AI-05).

What success looks like

- Drafts are real, grounded in the right reference chunks, and validated against DraftContent before they are stored.
- The medication table is rendered from the database; the model never writes a dose.
- An AI failure produces a 503 and a generation_failed draft, never a 500 or a half-written draft.
- You can show the retrieval results and the reading-grade improvement as evidence, not anecdotes.

Common pitfalls (Windows and macOS)

- The key reads as empty: load_dotenv() must run before os.environ is read, and uvicorn must be restarted after editing .env.
- A misspelled model name returns a 404 from the API. The fallback hides it, so always read the uvicorn console when a draft fails.
- Letting retrieval "fill in" patient facts: if a draft says the patient had three port-site cuts because the reference did, that is AI-03 broken. Tighten the system prompt and flag it for the reviewer.
- Forgetting max_tokens or the timeout leads to slow, costly calls. Keep them tight.

Go deeper • AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- Add prompt caching: send the stable system prompt as a content block with cache_control: {"type": "ephemeral"} and compare

`usage.cache_creation_input_tokens` and `usage.cache_read_input_tokens` across two calls. Note that caching only applies above a minimum prompt length.

- Replace JSON-in-text with forced tool use: define a `submit_draft` tool whose input schema is `DraftContent` and set `tool_choice` to that tool. Run 10 drafts each way and compare invalid-output rates.

Stretch goal

Swap `ANTHROPIC_MODEL` to `claude-haiku-4-5` and compare draft quality, reading grade, latency and tokens for the same record. Use `/cost` inside Claude Code for the build cost, and the stored token counts for the runtime cost. Decide which model you would use for drafting and which for a cheap second-opinion check.